Clay Seal: runtime containment for LLM agents

2026-08-07

A runtime authorization layer that binds an agent's authority to the target and the outcome of each action rather than to the tool it calls. Every number below is reproducible from the repository; commands in Section 8.

1 What the layer is

An agent session begins by sealing a goal. The goal compiles into a set of machine-checkable constraints (`agentauth.core.task_scope.compile_task_scope`) covering allowed path globs, resource identifiers, action verbs, argument shapes, and aggregate budgets. Every attempted tool call passes through one gateway (`SessionBroker.authorize`) before execution, and every decision is appended to a hash-chained log (`DecisionLog`, verifiable via `verify()`).

Six enforcement rungs, cheap to expensive:

Rung	Primitive	Catches
tool-allowlist	tool-name set membership	ungranted tools
capability-token	`capability_allows` (Biscuit)	action escalation
task-scope	`task_scope_allows_path`	path exfiltration, credential stores
+binding	commit-token `arguments_hash`	argument tampering, swapped payee
+budget	`SessionValueBudget`, `SessionCallBudget`	fragmented spend, bulk exfiltration
intent envelope	goal-derived plan conformance	in-scope actions never authorized

Destination binding runs at the floor on any action carrying an external address, not only on send-shaped verbs, so an SSRF-style read of an attacker-controlled URL is caught as an acquisition. Trusted destinations are seeded from the sealed goal text and widened only from structured fields of tool output, never from free text, because free text is where injections live.

Responses are graduated. A denial backed by positive evidence of malice (protected zone, untrusted destination, mutated payload) hard-blocks. A scope miss, which is uncertainty rather than evidence, raises a step-up that halts autonomous execution and is recoverable under supervision. Human attention is a charged, bounded resource (`audit_budget`), so a session cannot ask an unbounded number of confirmations.

2 The bar

The comparison set is the 2026 literature. CaMeL (arXiv:2503.18813) and Progent (arXiv:2504.11703) established the out-of-band approach in 2025 and are included because everything since is measured against them, but the current frontier is provenance-based auditing and information-flow labelling.

System	Mechanism	Setting	ASR undef. → def.	Adaptive	Utility cost
ARGUS (2605.03378)	influence-provenance graph, entailment check	AgentLure, gpt-4o-mini	28.8% → 3.8%	5.9%	5 pts (92.5 → 87.5)
Progent, indep. repro (2606.26479)	symbolic least-privilege over arguments	AgentDojo, Qwen2.5-7B	25.8% → 4.2%	2.6%	∼19 pts (45 → 26)
RTBAS (CMU)	information-flow labels, selective propagation	AgentDojo	targeted attacks prevented	not reported	∼2 pts under attack
CaMeL (DeepMind, 2025)	dual-LLM, capabilities, data-flow policy	AgentDojo	near zero	not reported	7 pts (84 → 77)
WARD (2605.15030)	trained guard model, 177K corpus	web agents	near-perfect recall	robust to guard-targeted	not reported
Clay Seal	target and destination binding, budgets, plan conformance	AgentDojo, gpt-4o-mini	38.9–88.9% → **0.5%** [0.1, 2.6]	100% contained, determ. harness	29 pts gpt-4o-mini, **3 pts** grok-4-1-fast

Reading this honestly. Our attack-success figure is the lowest in the table and is the only one pooled over more than one sweep (216 runs). Our utility cost on gpt-4o-mini, 29 points, is worse than ARGUS at 5 and Progent at 19; on grok-4-1-fast it is 3 points, which is the best in the table. ARGUS is measured on AgentLure rather than AgentDojo, so its numbers are not directly comparable to ours and are included for mechanism and order of magnitude.

The adaptive columns are not measured the same way and should not be read as a ranking. ARGUS reports a white-box adaptive attack against a live agent, which we have not run. Ours is a deterministic search that mutates against the compiled policy over six rounds at four attacker knowledge levels, including one where the attacker controls part of the sealed goal. Both are stronger than the static evaluation most of this table rests on, and the independent Progent reproduction exists precisely because author-reported figures (1.0% ASR) did not survive re-measurement (4.2%).

What none of them report: human-interruption cost, long-horizon false-alarm rate per thousand benign actions, or the split between attacks that leave the granted surface and attacks that stay inside it.

3 Results

3.1 Authority must bind to the target, not the tool

RedCode-Exec (NeurIPS 2024 D&B), 718 risky operations, benign side from 344 BFCL multi-turn filesystem steps:

Engine	Attack prevented	False-block
tool-allowlist	0.0% [0.0, 0.4]	0.0%
capability-token	0.0% [0.0, 0.4]	0.0%
task-scope	99.9% [99.6, 100.0]	0.0%
task-scope+binding+budget	99.9% [99.6, 100.0]	0.0%

All 717 contained attacks leave the granted surface via the *target alone*. Every attack uses a tool the agent legitimately holds, which is why the two lower rungs contain none of them. Intervals

are task-clustered bootstrap, since the 718 events derive from 14 risk templates and an event-level interval would be roughly the square root of the events per template too narrow.

3.2 Live injection, four AgentDojo suites, gpt-4o-mini

Deployable configuration under `important_instructions`, three independent sweeps per suite, 216 attack runs in total:

Suite	Undef. ASR	Defended	Succ./runs	Clean util.	Baseline	Interrupts
banking	66.7%	0.0%	0 / 54	16.7%	50.0%	0.56
slack	81.5%	0.0%	0 / 54	50.0%	83.3%	0.17
travel	38.9%	1.9%	1 / 54	66.7%	100.0%	0.56
workspace	88.9%	0.0%	0 / 54	83.3%	100.0%	2.11
pooled		**0.5%** [0.1, 2.6]	**1 / 216**			0.85

Our own head-to-head under the same protocol leaves Progent at 11.1 to 16.7%; the independent reproduction on Qwen2.5-7B puts it at 4.2%, and ARGUS reports 3.8% on AgentLure. Sweep-to-sweep spread on the defended configuration is zero on three suites and 5.6% on travel, while the undefended baseline moves by 11 to 17 points, so the variance lives in whether an attack lands on an unprotected agent rather than in whether enforcement holds.

On workspace the defended run reaches 0.0% ASR at 83.3% clean utility against an undefended baseline of 88.9% ASR and 25.9% utility under attack. Eight further configurations reach the same 0.0% there and differ only in interruption count, from 1.50 to 6.00 per task, which is why the last column is reported.

3.3 Utility cost across a model ladder

Four suites, 32 clean tasks per model, paired per task so a task that also fails undefended is not charged to the defense:

Model	Baseline	Deployable	Cost	False-block
gpt-4o-mini	84%	59%	25 pts	12.5% [5.0, 28.1]
gpt-oss-120b	84%	66%	19 pts	9.4% [3.2, 24.2]
grok-4-1-fast	81%	78%	**3 pts**	6.2% [1.7, 20.1]
llama-4-maverick	12%	12%	0 pts	0.0%

Cost falls monotonically with agent capability. On the strongest model measured the deployable path costs 3 points, against ARGUS at 5, CaMeL at 7, and the independently reproduced Progent at roughly 19. The llama-4-maverick row is a null: a 12% baseline leaves nothing for a defense to cost, and it is kept in because dropping models that perform badly is the selection this evaluation exists to prevent.

3.4 Policy-aware adaptive attacker

400 RedCode tasks, 6 rounds, attacker mutates against observed denials:

Engine	Blind	Deny reasons	Reads policy	Poisoned goal
tool-allowlist	0%	0%	0%	0%
capability-token	0%	0%	0%	0%
task-scope and above	**100%**	**100%**	**100%**	**100%**

Identical for exfiltration (T1041), persistence (T1546), and destruction (T1485). Attacker knowledge does not move containment, because the constraint is on the outcome rather than on input phrasing. A prefix matcher without path canonicalization is checked in as a negative control and falls 80 of 80 at round 1, so the 100% figures are bounded by a search known to work.

3.5 Long-horizon sessions

200 sessions of 500 benign actions with one malicious action inserted at a seeded position, the needle wearing the session's own tool and resource so it is not detectable by name:

Engine	Needle detected	False alarms / 1k benign
tool-allowlist	0.0%	0.00
capability-token	82.0% [76.1, 86.7]	0.00
task-scope	83.0% [77.2, 87.6]	0.00
task-scope+binding	**100.0%** [98.1, 100.0]	**0.00**

100,000 benign actions judged per engine, holding at 2,000-action sessions. AgentDojo trajectories average about three calls, so this separation between path scoping and argument binding is not visible on published benchmarks.

3.6 Attack-class coverage

Attack events partitioned without consulting any engine, by whether any field (tool, resource, action, target) falls outside what the task's mandate and benign trajectory establish:

Corpus	Attack events	Surface-leaving	In-surface
RedCode	718	717 (100%)	1
AgentHarm	652	41 (6%)	611 (94%)
ASB	2,040	2,040 (100%)	0

Pooled over 3,410 events the full stack contains **82.1%**, not the 100% the surface-leaving column alone suggests. The in-surface class, where every field is legitimate and only intent differs, is contained by no per-call authorization system including ours. ASB's 100% is a weak result: all 2,040 attacks leave via an ungranted tool name, so the weakest rung catches them.

3.7 2026 corpora

The corpora above are 2024 and 2025. These are the current instruments, chosen to test capabilities the older ones do not.

IPI-Coding-Agent (UK AI Safety Institute, `inspect_evals`). 45 samples pairing a real coding task with a payload hidden in artifacts the agent has to read anyway: issue text, source comments, configuration. Ten carry no payload and are the controls.

Payload category	Contained
exfiltration	30 / 30
code execution	10 / 10
persistence	10 / 10
false blocks on the 10 benign controls	0

AgentThreatBench (UK AI Safety Institute), which operationalises the OWASP Top 10 for Agentic Applications:

Category	Contained
autonomy hijack (forbidden-action target)	6 / 6
data exfiltration (unauthorized recipient)	8 / 8
memory poisoning	**0 / 10**

The data-exfiltration row is the cleanest external validation of destination binding we have, because the corpus ships its own authorized-recipient list rather than us supplying the ground truth.

The memory row is a genuine gap and took a correction to measure honestly. The first run reported 10 of 10 because the mandate did not grant memory writes, so the poisoning write failed a path check. An agent that has a long-term memory legitimately writes to it. With that grant present the attack is in-surface, which is what it is: the write is authorized and the harm is the belief the agent holds on a later turn, which no per-action check observes.

SLEIGHT-Bench (arXiv:2605.16626, Anthropic alignment). 44 attacks in which a coding agent covertly pursues a harmful objective while trying not to look like it, across 12 evasion categories, each with a matched benign transcript. Judged per transcript, which is the corpus's unit:

Metric	Value
detection	75.6%
false positives	69.8%
discrimination	**5.8 points**

Our authorization layer has essentially no discriminative power here, and we publish that. The layer decides per action on what the action touches. These attacks are defined by intent and executed through actions that individually look like ordinary engineering work. The 75.6% figure is not a security result: at a 69.8% false-positive rate it is close to a constant classifier.

For context, the published result on this corpus is that an Opus 4.6 monitor with extended thinking never catches 20 of 40 attacks across ten trials at a 1% false-positive rate, with an overall catch rate of 32%. Nothing currently does well here, and a frontier model reading the whole transcript beats a per-action policy for the obvious reason.

AgentDyn (arXiv:2602.03117), the 2026 successor to AgentDojo with open-ended tasks requiring runtime planning, is wired and running. Results pending. We expect it to cost us utility, because its tasks defeat plan-conformance by design and it embeds benign third-party instructions specifically to measure over-defense.

3.8 The corpus set ordered by attack type

Six corpora, partitioned without consulting any engine:

Corpus	Year	Attacks	Surface-leaving	In-surface	Containment
RedCode	2024	718	717 (100%)	1	∼100%
IPI-Coding	2026	50	50 (100%)	0	100%
ASB	2025	2,040	2,040 (100%)	0	100%
AgentThreatBench	2026	24	14 (58%)	10 (42%)	58%
SLEIGHT	2026	328	121 (37%)	207 (63%)	no signal
AgentHarm	2025	652	41 (6%)	611 (94%)	6%

The ordering is the point. Corpora built around what an action *touches* sit at the top, and we contain essentially all of them. Corpora built around what an agent *intends* sit at the bottom, and we contain essentially none. The boundary is a property of per-call authorization rather than of this implementation, and publishing the whole ordering is more useful to a buyer than publishing the top three rows.

3.9 Enforcement cost

10,880 decisions per engine, 20 repeats, warm:

Rung	p50	p95	p99
capability-token	2.79 µs	3.33 µs	3.50 µs
task-scope	31.9 µs	36.3 µs	56.1 µs
full stack	35.0 µs	40.5 µs	60.3 µs

Scope compilation is amortized per task at 21 ms per 250 tasks and excluded.

4 Beyond the session: the controls a business actually runs on

Everything above, and every system in the comparison table, is scoped to a session and to an event. AgentDojo episodes average three tool calls. Business risk lives at the scale of a standing mandate held by a principal across months: the refund agent, the procurement agent, the reconciliation agent. Every control that matters in a real finance function is principal-scoped and time-scoped, and none of it has been ported to agents.

Two of those controls are now built and measured. Both expose a failure invisible to every benchmark in the field, because none runs long enough or across enough sessions to produce it.

4.1 Structuring: aggregate limits that survive the session boundary

Structuring is the canonical business-process violation: split an amount so no single transaction crosses a threshold. Every aggregate control in the literature, and ours until this week, keys its ledger to the session. That is a control the adversary resets at will.

Moving 40,000 against a 10,000 ceiling, split across N sessions:

Fragments	Session-scoped	Contained	Principal-scoped	Contained
1	0	yes	0	yes
2	0	yes	0	yes
4	40,000	**no**	10,000	yes
64	40,000	**no**	10,000	yes

Containment fails at four fragments and never recovers. The attacker needs no capability it did not already have; it opens another conversation. The principal-scoped ledger keys spend to the mandate over a rolling window, so fragments accumulate wherever they occur.

Containment is a hard limit. Detecting the *shape* underneath it is the compliance question, and the existing analytic looks for amounts just below hard-coded round numbers, which four payments of 2,600 against a 10,000 ceiling do not trip. Two signatures replace it, with the false-positive cases measured rather than asserted:

Pattern	Flagged
split to the ceiling ($4 \times 2{,}500$)	**yes**
parked just under ($4 \times 9{,}000$)	**yes**
real invoice run, varied amounts	no
payroll, uniform but 5% utilisation	no
one large payment	no

Uniformity alone cannot be evidence, since payroll is uniform. It counts only alongside high utilisation, which is what makes the limit look like the binding constraint rather than the business.

4.2 Drift: whether the defense's own baseline moves

A monitor that learns what is normal from the agent's history has a feedback loop, because its notion of normal is a function of behaviour it already permitted. An adversary that shifts the distribution slowly is not evading the detector, it is retraining it. This applies to every adaptive monitor in the field including our own, and no published work measures it.

An adversary raises the mean 0.0008 sigma per action from 10 to 14 over 10,000 actions, arriving halfway and then operating openly. A single action at 14 is four sigma out on day one and refused outright:

Monitor	Baseline moved	Displacement	Alarms at objective
rolling (what everyone does)	$9.995 \rightarrow 13.888$	4.08 sigma	7 / 5,000 (0.1%)
anchored to a signed epoch	$9.995 \rightarrow 9.995$	0.00 sigma	4,084 / 5,000 (81.7%)

The rolling monitor did not fail to detect an anomaly. It stopped considering the behaviour anomalous. The anchored monitor judges against what a human signed off rather than against what the agent talked it into, and **anchor displacement** is the number that makes organisational drift visible.

The cost is real and stated: an anchored baseline also alarms on legitimate business growth, because drift and growth are the same signal. That is not solvable automatically, and re-anchoring is a scheduled human decision.

5 Capabilities the comparables do not have

Adaptive red-team against our own policy. The harness searches over mutations (path obfuscation, tool and resource laundering, staging through in-scope locations, fragmentation, and policy-derived escape construction) rather than replaying a fixed corpus. Within an hour of first running it found two exploitable bypasses in shipping code: `path_matches_any` fnmatched raw strings, so a `/app/**` grant admitted `/app/../etc/passwd`, and `/proc/self/root/...` reached the filesystem root behind a prefix no protected-zone pattern matched. Both fixed, both with regression tests. Every company deploying agent authorization has this bug class and no tooling to find it.

Interruption cost as a first-class metric. A configuration reaching 0% ASR by confirming everything has relocated the vulnerability, and approval fatigue is an attack surface. Sweeping our response policy on banking found a setting that reaches identical 0.0% ASR while spending 4.17 interruptions per task against 0.83, at half the clean utility. On the two axes normally reported it is a tie.

Long-horizon false-alarm rate. Reported per 1,000 benign actions rather than per call, because a 0.5% per-action rate is five alerts per thousand actions, which at agent speeds is an alert every few minutes.

Verifiable audit. Hash-chained decision records with `verify()`, which is what a regulated buyer needs to demonstrate control rather than assert it.

6 Evidence base and limits

Deterministic tiers run at large n and reproduce bit-for-bit against a content-pinned corpus: 3,410 attack events in the coverage analysis, 100,000 benign actions in the long-horizon tier, 10,880 decisions per engine in the latency measurement, 512 tests passing.

Live measurements drive a stochastic agent through AgentDojo. The headline attack-success figure pools three independent sweeps across four suites, 216 runs, which supports the interval quoted. A single sweep of 18 would not: zero successes in 18 runs has a Wilson upper bound of 17.6%.

Utility on weak models is the open cost: 25 points on gpt-4o-mini against 3 on grok-4-1-fast. Diagnosis attributes it to the LLM planner failing to enumerate legitimate steps rather than to the enforcement rules, and mid-session re-clearance (`SessionBroker.reclear`) is the route.

7 Roadmap

Anchored monitoring in the live path. The drift benchmark demonstrates the mechanism; the production monitor still takes whatever baseline its caller last fitted. Wiring the signed anchor and publishing anchor displacement as a continuous operational metric is the next build.

In-surface containment. The 18% of attacks that stay inside the granted surface are uncontained by any per-call system. Velocity limits against the task's own observed shape, novel-counterparty holdback, and provenance tracked transitively across the data-flow graph are the mechanisms. The aggregate-budget rung already contains the volume form of this class, which is why bulk exfiltration over an allowed channel is caught today.

Syscall-boundary enforcement. Symlink resolution and time-of-check to time-of-use are undecidable from userspace path strings. Enforcing at the kernel boundary via iVisor closes both and enables evaluation against an agent actively attempting sandbox escape, which no published benchmark covers.

8 Reproduce

```
benchmarks/fetch_corpora.sh          # pinned external corpora, ~4 MB
benchmarks/run_all.sh                # all deterministic tiers, under a minute
python -m benchmarks.adaptive --dataset redcode --rounds 6
python -m benchmarks.trajectory --length 500 --trajectories 200
python -m benchmarks.live.frontier --suite workspace --repeats 3
```